

Laboratorio #6 – Manejo de errores

Nombre	Fer Guzman	Fecha	08/10/2026
ID	[Completar]	Carrera	[Completar]

Introducción

En este laboratorio se aplicó manejo de excepciones y pruebas unitarias con pytest sobre la clase ExceptionsDemo desarrollada en Python. Se trabajó en detectar y manejar errores usando try-except, validar métodos individuales ante entradas inválidas, y verificar que el sistema maneje correctamente las excepciones.

Repositorio del laboratorio

Fork en GitHub: <https://github.com/FerGuzman65/pruebas-de-software-a-lab-6-exepciones-Lab6>

Actividad 1: Excepciones

Se abrió el archivo test_exceptions.py y se ejecutó pytest en la terminal. Las pruebas originales (con pytest.raises) verifican que ZeroDivisionError, TypeError, IndexError, KeyError, FileNotFoundError, AttributeError y MyException se generen correctamente al llamar los métodos de ExceptionsDemo con entradas inválidas.

Actividad 2: Excepciones

Se modificó cada método de la clase ExceptionsDemo (Exceptions.py) para incorporar bloques try-except, de manera que ante cualquier excepción el programa no se detenga y el flujo continúe de forma controlada, imprimiendo un mensaje de error en vez de propagar la excepción.

¿Qué excepciones se agregaron a cada función?

Método	Excepción(es) capturada(s)
divide(a, b)	ZeroDivisionError, TypeError
access_list(lst, index)	IndexError
access_dict(dic, key)	KeyError
read_file(filename)	FileNotFoundError, IOError
access_attribute(obj)	AttributeError
check_positive(number)	MyException (personalizada)

Análisis de los errores encontrados

Al completar la Actividad 2 y volver a ejecutar las pruebas originales de la Actividad 1 (las que usan `pytest.raises`), las 7 pruebas fallan con el mensaje 'DID NOT RAISE'. Esto ocurre porque, una vez que cada método de `ExceptionsDemo` captura su excepción internamente con `try-except`, la excepción ya no se propaga hacia afuera del método; en su lugar, el método imprime un mensaje de error y continúa. Como `pytest.raises` espera que la excepción efectivamente se lance fuera del bloque `with`, al no lanzarse (porque fue manejada internamente) la prueba se marca como fallida, aunque el comportamiento del programa es el correcto según lo pedido en la Actividad 2 (no detenerse ante errores). En la salida capturada de cada prueba (ver Anexos) se puede confirmar que el mensaje de error esperado sí se imprimió correctamente en cada caso.

Anexos – Capturas de pantalla

Exceptions.py (clase `ExceptionsDemo` con manejo de excepciones):

Exceptions.py

```
1  class MyException(Exception):
2      pass
3
4  class ExceptionsDemo:
5
6      def divide(self, a, b):
7          try:
8              result = a / b
9              print("Division result:", result)
10         except ZeroDivisionError:
11             print("Error: Division by zero is not allowed.")
12         except TypeError:
13             print("Error: Invalid type for division. Please provide numbers.")
14
15
16
17     def access_list(self, lst, index):
18         try:
19             print("Element:", lst[index])
20         except IndexError:
21             print("Error: Index is out of bounds.")
22
23
24     def access_dict(self, dic, key):
25         try:
26             print("Value:", dic[key])
27         except KeyError:
28             print("Error: Key does not exist in the dictionary.")
29
30     def read_file(self, filename):
31         try:
32             with open(filename, "r") as f:
33                 content = f.read()
34                 print(content)
35         except FileNotFoundError:
36             print("Error: File not found.")
37         except IOError:
38             print("Error: Could not read the file.")
39
40     def access_attribute(self, obj):
41         try:
42             print(obj.nonexistent_attribute)
43         except AttributeError:
44             print("Error: The object does not have that attribute.")
45
46     def check_positive(self, number):
47         try:
48             if number < 0:
49                 raise MyException("The number must be positive.")
50             else:
51                 print("Valid number:", number)
52         except MyException as e:
53             print(f"Error: {e}")
```

test_exceptions.py:

```
test_exceptions.py
1  import pytest
2  from Exceptions import ExceptionsDemo, MyException
3
4
5
6  def test_division_cero():
7      demo = ExceptionsDemo()
8      with pytest.raises(ZeroDivisionError):
9          demo.divide(10, 0)
10
11
12
13  def test_division_with_string ():
14      demo=ExceptionsDemo()
15      with pytest.raises(TypeError):
16          demo.divide(10, "string")
17
18
19  def test_access_list_out():
20      demo=ExceptionsDemo()
21      with pytest.raises(IndexError):
22          demo.access_list([1,3,4], 7)
23
24  def test_access_dic():
25      demo=ExceptionsDemo()
26      with pytest.raises(KeyError):
27          demo.access_dict({"color": "red", "forma": "cuadrada"}, "sabor")
28
29
30  def test_read_file():
31      demo=ExceptionsDemo()
32      with pytest.raises(FileNotFoundError):
33          demo.read_file("texts.txt")
34
35  def test_access_attribute():
36      demo=ExceptionsDemo()
37      with pytest.raises(AttributeError):
38          demo.access_attribute([1,3,4])
39
40
41  def test_check_positive():
42      demo=ExceptionsDemo()
43      with pytest.raises(MyException):
44          demo.check_positive(-5)
```

Resultado de pytest -v:

pytest -v (Lab 6)

```

1  ===== test session starts =====
2  platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\manue\AppData\Local\Programs\Python\Python39\python.exe
3  cachedir: .pytest_cache
4  rootdir: C:\Users\manue\OneDrive - Universidad Autonoma de Guadalajara\UAG\Pruebas de software\Parcial 2\Lab 6
5  collecting ... collected 7 items
6
7  test_exceptions.py::test_division_cero FAILED [ 14%]
8  test_exceptions.py::test_division_with_string FAILED [ 28%]
9  test_exceptions.py::test_access_list_out FAILED [ 42%]
10 test_exceptions.py::test_access_dic FAILED [ 57%]
11 test_exceptions.py::test_read_file FAILED [ 71%]
12 test_exceptions.py::test_access_attribute FAILED [ 85%]
13 test_exceptions.py::test_check_positive FAILED [100%]
14
15 ===== FAILURES =====
16 _____ test_division_cero _____
17
18     def test_division_cero():
19         demo = ExceptionsDemo()
20         with pytest.raises(ZeroDivisionError):
21             > demo.divide(10, 0)
22     E       Failed: DID NOT RAISE <class 'ZeroDivisionError'>
23
24 test_exceptions.py:9: Failed
25 ----- Captured stdout call -----
26 Error: Division by zero is not allowed.
27 _____ test_division_with_string _____
28
29     def test_division_with_string ():
30         demo=ExceptionsDemo()
31         with pytest.raises(TypeError):
32             > demo.divide(10, "string")
33     E       Failed: DID NOT RAISE <class 'TypeError'>
34
35 test_exceptions.py:16: Failed
36 ----- Captured stdout call -----
37 Error: Invalid type for division. Please provide numbers.
38 _____ test_access_list_out _____
39
40     def test_access_list_out():
41         demo=ExceptionsDemo()
42         with pytest.raises(IndexError):
43             > demo.access_list([1,3,4], 7)
44     E       Failed: DID NOT RAISE <class 'IndexError'>
45
46 test_exceptions.py:22: Failed
47 ----- Captured stdout call -----
48 Error: Index is out of bounds.
49 _____ test_access_dic _____
50
51     def test_access_dic():
52         demo=ExceptionsDemo()
53         with pytest.raises(KeyError):
54             > demo.access_dict({"color": "red", "forma": "cuadrada"}, "sabor")
55     E       Failed: DID NOT RAISE <class 'KeyError'>
56
57 test_exceptions.py:27: Failed
58 ----- Captured stdout call -----
59 Error: Key does not exist in the dictionary.
60 _____ test_read_file _____
61
62     def test_read_file():
63         demo=ExceptionsDemo()
64         with pytest.raises(FileNotFoundError):
65             > demo.read_file("texts.txt")
66     E       Failed: DID NOT RAISE <class 'FileNotFoundError'>
67
68 test_exceptions.py:33: Failed
69 ----- Captured stdout call -----
70 Error: File not found.
71 _____ test_access_attribute _____
72
73     def test_access_attribute():
74         demo=ExceptionsDemo()
75         with pytest.raises(AttributeError):
76             > demo.access_attribute([1,3,4])
77     E       Failed: DID NOT RAISE <class 'AttributeError'>
78
79 test_exceptions.py:38: Failed
80 ----- Captured stdout call -----
81 Error: The object does not have that attribute.
82 _____ test_check_positive _____
83
84     def test_check_positive():
85         demo=ExceptionsDemo()
86         with pytest.raises(MyException):
87             > demo.check_positive(-5)
88     E       Failed: DID NOT RAISE <class 'Exceptions.MyException'>
89
90 test_exceptions.py:44: Failed
91 ----- Captured stdout call -----
92 Error: The number must be positive.

```

Conclusiones

Este laboratorio permitió comprender la diferencia entre dejar que una excepción se propague (para que algo externo, como una prueba con `pytest.raises`, la detecte) y manejarla internamente con `try-except` (para que el programa no se detenga). Ambos enfoques son válidos pero sirven a propósitos distintos: el primero es útil para verificar que el código efectivamente detecta condiciones de error, mientras que el segundo es el comportamiento deseado en un programa en producción, donde no queremos que un error inesperado detenga la ejecución completa. La lección principal es que agregar manejo de errores a una función cambia su comportamiento observable, por lo que las pruebas que dependen de ese comportamiento también deben revisarse y actualizarse en consecuencia.